

Outros conceitos de Kotlin

MOV786205 – CST em Análise e Desenvolvimento de Sistemas

Prof. Emerson Ribeiro de Mello

mello@ifsc.edu.br

Licenciamento



Slides licenciados sob [Creative Commons "Atribuição 4.0 Internacional"](#)

Sumário

- 1 Enumerações e tipos genéricos
- 2 Tratamento de exceções
- 3 Extensão de funções
- 4 Data class
- 5 Singleton
- 6 Companion object
- 7 Declaração desestruturadas
- 8 Delegação de propriedades
- 9 Coroutines

Enumerações e tipos genéricos

Enumerações

```
package enumeracoes

enum class DiaDaSemana(val dia: String) {
    DOMINGO("Domingo"),
    SEGUNDA("Segunda-feira"),
    TERÇA("Terça-feira"),
    QUARTA("Quarta-feira"),
    QUINTA("Quinta-feira"),
    SEXTA("Sexta-feira"),
    SÁBADO("Sábado");

    fun descricao() = "Hoje é $dia"
}

fun main() {
    val hoje = DiaDaSemana.SEGUNDA
    println(hoje.descricao())
}
```

Tipos genéricos

```
package genericos

class Caixa<T> (var conteudo: T)

fun main() {
    // Informando o tipo de forma explícita
    val caixaInt = Caixa<Int>(123)

    // Inferindo o tipo automaticamente
    val caixaString = Caixa("Olá, mundo!")

    println(caixaInt.conteudo)
    println(caixaString.conteudo)
}
```

Tratamento de exceções

Tratamento de exceções

- Tratamento de exceções em Kotlin é similar ao Java
- Usa-se `try`, `catch` e `finally`
- Não existem exceções verificadas (*checked exceptions*), assim não é necessário declarar exceções em funções

Exemplo de tratamento de exceções

```
package tratamento

fun main() {
    try {
        val resultado = 10 / 0
        println("Resultado: $resultado")
    } catch (e: ArithmeticException) {
        println("Erro: ${e.message}")
    } finally {
        println("Bloco finally executado")
    }
}
```

Exceções personalizadas

```
package tratamento

class DivisaoPorZeroException(mensagem: String) : Exception(mensagem)

fun main() {
    try {
        val resultado = dividir(10, 0)
        println("Resultado: $resultado")
    } catch (e: DivisaoPorZeroException) {
        println("Erro: ${e.message}")
    }
}

fun dividir(a: Int, b: Int): Int {
    if (b == 0) {
        throw DivisaoPorZeroException("Divisão por zero não é permitida")
    }
    return a / b
}
```

Funções de pré-condição: require

- Se a condição for falsa, lança uma exceção `IllegalArgumentException`
- Torna o código mais legível e seguro

```
fun calcularIdade(anoNascimento: Int) {  
    require(anoNascimento > 0) { "Ano de nascimento inválido" }  
    val idade = 2025 - anoNascimento  
    println("Idade: $idade")  
}  
  
fun main() {  
    calcularIdade(1990)  
    calcularIdade(-5)  
}
```

Extensão de funções

Extensão de funções

- Permite adicionar novas funcionalidades a classes existentes sem a necessidade de herança
- Útil para adicionar métodos a classes de bibliotecas ou APIs sem modificá-las

```
// Função de extensão para a classe Int
fun Int.isPar(): Boolean {
    return this % 2 == 0
}

fun main() {
    val numero = 4
    if (numero.isPar()) {
        println("$numero é par")
    } else {
        println("$numero é ímpar")
    }
}
```

Extensão de propriedades

- Permite adicionar novas propriedades a classes existentes

```
class Pessoa(var nome: String, var sobrenome: String)

// Propriedade de extensão para a classe Pessoa
val Pessoa.nomeCompleto: String
    get() = "$nome $sobrenome"

fun main() {
    val pessoa = Pessoa("Juca", "Silva")
    println("Nome completo: ${pessoa.nomeCompleto}")
}
```

Data class

Data class

- Representam classes para armazenar dados, gerando automaticamente métodos como `equals()`, `hashCode()`, `toString()` e `copy()`.
- Semelhantes aos `record` do Java, mas permitem atributos mutáveis ou imutáveis.

```
data class Usuario(val id: Int, val nome: String)

fun main() {
    val usuario1 = Usuario(1, "Alice")
    val usuario2 = Usuario(1, "Alice")

    println(usuario1) // Usuario(id=1, nome=Alice)
    println(usuario1 == usuario2) // true
    val usuario3 = usuario1.copy(nome = "Bob")
    println(usuario3) // Usuario(id=1, nome=Bob)
}
```


Sealed Classes

- Sealed classes representam hierarquias restritas de tipos relacionados
- Permitem definir uma classe base com um conjunto fixo de subclasses
- Úteis para modelar estados ou resultados previsíveis e seguros
- Declaradas com `sealed`; subclasses devem estar no mesmo arquivo

Exemplo de Sealed Class

```
sealed class Resultado{  
    data class Sucesso(val dados: String) : Resultado()  
    data class Erro(val mensagem: String) : Resultado()  
}  
  
fun processarResultado(resultado: Resultado) {  
    when (resultado) {  
        is Sucesso -> println("Dados recebidos: ${resultado.dados}")  
        is Erro -> println("Erro: ${resultado.mensagem}")  
    }  
}  
  
fun main() {  
    val sucesso = Sucesso("Dados carregados com sucesso")  
    val erro = Erro("Falha ao carregar os dados")  
  
    processarResultado(sucesso)  
    processarResultado(erro)  
}
```

Singleton

Singleton

- Padrão de projeto que garante que uma classe tenha apenas uma instância e fornece um ponto de acesso global a essa instância
- Em Kotlin, pode ser implementado usando o objeto `object`

```
object Configuracao {  
    var url: String = "http://localhost"  
    var porta: Int = 8080  
  
    fun exibirConfiguracao() {  
        println("URL: $url, Porta: $porta")  
    }  
}  
  
fun main() {  
    Configuracao.exibirConfiguracao()  
    Configuracao.url = "http://example.com"  
    Configuracao.exibirConfiguracao()  
}
```

Companion object

Companion object

- Permite associar objetos a uma classe, semelhante a static em Java
- Útil para fábricas de objetos e funções utilitárias

```
class Calculadora {  
    fun somar(a: Int, b: Int): Int {  
        memoria+=(a+b)  
        return a + b  
    }  
  
    fun zerar(){  
        memoria = 0.0  
    }  
  
    companion object Funcionalidade {  
        var memoria = 0.0  
        private set  
    }  
}
```

```
fun main() {  
  
    val calc = Calculadora()  
  
    val res = calc.somar(1,2)  
  
    // Acessando a memória através do  
    // companion object  
    val m = Calculadora.memoria  
  
    calc.zerar()  
}
```

Declaração desestruturadas

Declaração desestruturadas

- Permite extrair valores de uma classe ou estrutura em variáveis separadas
- Isso facilita o acesso aos valores sem precisar acessar os propriedades diretamente
- Para funcionar, a classe deve ter métodos `componentN()` (gerados automaticamente em *data classes*)
- Também é possível implementar manualmente `operator fun componentN()` em suas próprias classes

Exemplo de declaração desestruturada

- A classe `Ponto` é uma *data class* com dois atributos: `x` e `y`
- Ao criar uma instância de `Ponto`, podemos desestruturar seus valores diretamente em variáveis `x` e `y`
- A desestruturação é especialmente útil quando trabalhamos com coleções de objetos, como listas ou mapas, onde podemos extrair facilmente os valores de cada objeto

```
data class Ponto(val x: Int, val y: Int)

fun main() {
    val ponto = Ponto(10, 20)
    val (x, y) = ponto
    println("X: $x, Y: $y")
}
```

Classes Pair e Triple

- Kotlin fornece classes auxiliares para trabalhar com pares e trios de valores
- `Pair` e `Triple` são usadas para armazenar dois ou três valores, respectivamente
- Essas classes são úteis para retornar múltiplos valores de uma função

Exemplo de Pair e Triple

```
fun dividir(x: Int, y: Int): Pair<Int, Int> {  
    return Pair(x / y, x % y)  
}  
  
fun main() {  
    val par = Pair("Kotlin", 2.2)  
    val (nome, versao) = par  
  
    println("Linguagem: $nome, Versão: $versao")  
  
    val triplo = Triple("Kotlin", "Java", "Scala")  
    val (linguagem1, linguagem2, linguagem3) = triplo  
  
    println("Linguagens: $linguagem1, $linguagem2, $linguagem3")  
  
    val (quociente, resto) = dividir(10, 3)  
    println("Quociente: $quociente, Resto: $resto")  
}
```

Delegação de propriedades

Delegação de propriedades

- Permite que a lógica de acesso e modificação de uma propriedade seja delegada a outro objeto, chamado de *delegado*
- Facilita a reutilização de comportamentos comuns, como propriedades preguiçosas (*lazy*), observáveis ou armazenamento personalizado
- Utiliza o operador **by** para indicar que a implementação da propriedade será fornecida por um delegado

Exemplo de delegação de propriedades

```
val mensagem: String by lazy {  
    println("Inicializando a mensagem...")  
    "Olá, Kotlin!" // Valor inicial  
}  
  
fun main() {  
    println("Antes de acessar a mensagem")  
    println(mensagem) // A inicialização ocorre aqui  
    println(mensagem) // Aqui não inicializa novamente  
}
```

Exemplo de delegação com classe

```
interface Voador {  
    fun voar()  
}  
  
class Passaro : Voador {  
    override fun voar() {  
        println("O pássaro está voando")  
    }  
}  
  
// delegou a implementação de Voador para o objeto voador  
class Passarinho(private val voador: Voador) : Voador by voador {  
    fun cantar() {  
        println("O passarinho está cantando")  
    }  
}  
  
fun main() {  
    val passarinho = Passarinho(Passaro())  
    passarinho.voar() // Delegado para a implementação de Passaro  
    passarinho.cantar() // Método específico de Passarinho  
}
```

Coroutines

Coroutines

- Coroutines são uma forma de programação assíncrona em Kotlin e usadas para operações que podem demorar, como acesso à rede ou banco de dados
- Utilizam funções *suspend*, *launch* OU *async*
 - Use *suspend* para funções que podem ser pausadas
 - Use *launch* para tarefas assíncronas sem retorno
 - Use *async* para tarefas assíncronas que retornam resultado

Principais conceitos de Coroutines

■ Coroutine Scope

- Controla o ciclo de vida das coroutines, garantindo que sejam canceladas ou finalizadas corretamente: `runBlocking` e `coroutineScope`

■ Dispatcher

- Define em qual thread a coroutine será executada, `Dispatchers.Main` (UI), `Dispatchers.IO` (I/O), `Dispatchers.Default` (CPU)

■ Job

- Representa uma coroutine ativa, permitindo controlar sua execução (cancelar, aguardar, etc.)

■ Suspend Function

- Função que pode ser pausada e retomada sem bloquear a thread

Exemplo básico de coroutine

- Coroutines não fazem parte da biblioteca padrão do Kotlin

```
// Para usar coroutines, adicione a seguinte dependência no arquivo: build.gradle.kts  
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.9.0")
```

```
import kotlinx.coroutines.*  
  
fun main(){  
    runBlocking { // Inicia um escopo de coroutine  
        println("Total de pedidos")  
        delay(1000L) // Só pode ser chamada dentro de uma coroutine  
        println("10 pizzas")  
    }  
}
```

- `runBlocking()` é síncrona e não irá retornar até todo o bloco da lambda ser executado

Funções suspend

Funções que podem ser pausadas sem bloquear a thread principal

```
// Como delay é uma suspend function, pode ser chamada dentro de outra suspend function
suspend fun obterPedidosDePizza() {
    delay(2000L)
    println("10 pizzas")
}

suspend fun obterPedidosDeBebidas(){
    delay(2000L)
    println("5 bebidas")
}

fun main() {
    val tempoGasto = measureTime { // para medir o tempo de execução
        runBlocking { // espera todo o bloco ser executado
            println("Total de pedidos")
            obterPedidosDePizza()
            obterPedidosDeBebidas()
        }
    }
    println("Tempo de execução: ${tempoGasto/1000.0} segundos")
}
```

Código assíncrono com launch

- launch inicia uma nova coroutine sem bloquear a thread atual, permitindo que outras tarefas sejam executadas concorrentemente

```
fun main() {  
    val tempoGasto = measureTime {  
        runBlocking {  
            println("Total de pedidos")  
            launch { // Inicia uma nova coroutine  
                obterPedidosDePizza()  
            }  
            launch {  
                obterPedidosDeBebidas()  
            }  
            println("Aguardando pedidos...") // Executada imediatamente  
        }  
    }  
    println("Tempo de execução: ${tempoGasto/1000.0} segundos")  
}
```

Obtendo resultado com `async` e `await`

- No exemplo anterior com `launch`, não era possível obter um resultado diretamente da coroutine
- `async` é semelhante a `launch`, mas retorna um objeto `Deferred` que pode ser usado para obter o resultado da coroutine
- `await` é usado para obter o resultado de uma coroutine iniciada com `async`
- Se for desejado exibir um resultado compilando os valores quando ambas as coroutines forem concluídas, `async` e `await` são as ferramentas certas a serem usadas

```

suspend fun obterPedidosDePizza() : String {
    delay(2000L)
    return "10 pizzas"
}

suspend fun obterPedidosDeBebidas() : String{
    delay(2000L)
    return "5 bebidas"
}

fun main() {
    val tempoGasto = measureTime {
        runBlocking {
            println("Total de pedidos")
            val pizza: Deferred<String> = async {
                obterPedidosDePizza()
            }
            val bebidas = async{ // inferindo o tipo
                obterPedidosDeBebidas()
            }
            println("Aguardando pedidos...") // Executada imediatamente
            println("${pizza.await()}, ${bebidas.await()}") // Aguarda os resultados
        }
    }
    println("Tempo de execução: ${tempoGasto/1000.0} segundos")
}

```

Decomposição com coroutineScope

- O uso de coroutineScope permite estruturar melhor o código assíncrono, aguardando a conclusão de todas as coroutines filhas antes de prosseguir.

```
suspend fun obterPedidos() = coroutineScope {  
    val pizza = async { obterPedidosDePizza() }  
    val bebidas = async { obterPedidosDeBebidas() }  
    "${pizza.await()}, ${bebidas.await()}" // retorno  
}  
  
fun main() {  
    val tempoGasto = measureTime {  
        runBlocking {  
            println("Total de pedidos")  
            println(obterPedidos())  
            println("Obrigado!") // Executada após a conclusão das tarefas  
        }  
    }  
    println("Tempo de execução: ${tempoGasto/1000.0} segundos")  
}
```


Aula baseada em



JETBRAINS. **Kotlin Programming Language**. 2025. Disponível em:
<https://kotlinlang.org/docs/>. Acesso em: 9 jul. 2025.